Anuj Pratap Singh Yadav

linkedin.com/in/yadavaps77 | +91-97523-73677 | Noida, India | yadav.anuj.singh@gmail.com

SUMMARY

Staff engineer with 11 years of building large, long-lived C++ systems at Synopsys, Microsoft and Adobe. I own coverage-analytics subsystems in a verification product end to end, lead architecture changes across team boundaries in a large C++ codebase, and build LLVM-based static analysis.

SKILLS

Languages: C++, Python, Tcl, C#, Java, TypeScript

AI-assisted engineering: Cursor agent skills, rules and subagents built for the team's workflow, shared with the team

Architecture and systems: large-codebase dependency management (dependency inversion, IWYU), LLVM IR passes and static analysis, master/worker job orchestration, license management, serialization (XML, pugixml)

Tools: GDB, Linux, LSF, Perforce, Git, Win32

Domain: verification tooling, coverage analytics

EXPERIENCE

R&D Staff Engineer | Synopsys *Noida, India | May 2023 - Present*

VC Formal, a verification product for chip designs; my app checks a C++ model against its hardware (RTL) implementation

- **Large-codebase architecture: build-dependency decoupling (multi-org initiative)**: led VC Formal's part. The shared C++ layer under every app included engine headers, so one engine edit rebuilt every app. Cut wrong-way dependencies ~73% with include-what-you-use, hide-what-you-use (forward declarations), dependency inversion through startup-injected callbacks, and relocating misplaced code, rolled out in small reviewed changes to another team's code and shipped to production. Shared the design with the wider org in a tech talk.
- **Coverage analytics for C++ code**: own it end to end. It shows which parts of a C++ model the automated proofs actually exercised, so C++ can be signed off like hardware code. Added merging across many runs and export to the standard coverage database, so C++ and hardware coverage are reviewed together in one viewer; drive changes with the viewer and GUI teams, handle customer issues, and guided its integration into a new sign-off app.
- **LLVM static analysis (customer-requested R&D)**: building program slicing that reports the exact C++ source lines a checked property depends on. Built on the open-source dg LLVM slicer: wrote LLVM IR passes for C++ constructs it missed and a signal-based slicing criterion that cut whole-file slices down to the lines feeding each signal. Design reviewed weekly with a Distinguished Engineer.
- **Distributed job orchestration: license management**: built token licensing in the C++ engine and moved runtime license checkout from workers to the master of a master/worker job runner, so one process owns keys for every tier and each key serves more workers; fixed production checkout failures across license servers.
- **Coverage analytics for hardware designs**: own property density and RTL coverage, which report which signals, down to the bit, and which lines the checks actually exercise. Rebuilt the density analysis on a shared design model so both apps give the same answer, adding bit-level detail; aligned coverage semantics across both. In production.
- **Session persistence (serialization)**: extended snapshots to capture state that was silently lost on restore (user Tcl procedures, engine state as XML), and kept new features' state persistent with round-trip tests.

Software Engineer II | Microsoft *Noida, India | May 2021 - May 2023*

- Owned Recording Studio features in PowerPoint Win32 (C++): camera modes, live background blur, video export and Cameo, a camera object placed on the slide. Led v1 test automation and wired Recording and Export telemetry.
- Designed catalog filtering plus an allowlist (C#) so new commerce SKUs no longer broke OneDrive's upgrade path.
- Built the Stream 2.0 video playback component in Excel Win32 (C++, TypeScript) for SharePoint and OneDrive video.

Software Development Engineer 2 | Adobe *Noida, India | Aug 2015 - Apr 2021*

Software Engineer Aug 2015 - Jan 2017; SDE 2 from Jan 2017

- Moved Photoshop Elements and Premiere Elements from RIBS to Hyperdrive builds with parallel install bundles, cutting install time ~50% on average. Spot Award.
- Designed lazy loading for large files in the AEM web editor, cutting load time ~60% on files customers reported as freezing.

EDUCATION

B.Tech, Computer Science and Engineering - NIT Bhopal (MANIT) | 2011 - 2015 | GPA 8.27/10